Automated Alert Triage & Enrichment Pipeline

SOC Lab Project Documentation (Project 2) — a lightweight SOAR-style pipeline on top of Wazuh

Author: Sawhard Poojari | Date: 23 October 2026 | Environment: Home SOC Lab (isolated)

---

1. Introduction

In a real Security Operations Center, every alert a SIEM raises needs context before an analyst can act on it: Is that IP known-bad? Is that file malware? Who is the account involved? Gathering this context by hand for every alert is slow and repetitive, and it is the first thing mature SOC teams automate.

This project builds that automation. On top of the Wazuh SIEM from Project 1, I designed and built an alert triage and enrichment pipeline: when a selected alert fires, a script automatically extracts its key indicators, looks them up against threat-intelligence services, assigns a verdict and priority, and posts a clean, enriched summary to a Discord channel. It is a lightweight, self-built version of what commercial SOAR platforms do.

2. Objectives

- Automatically enrich Wazuh alerts with external threat intelligence (IP reputation, file-hash reputation, geolocation).
- Assign each alert a clear verdict (Informational / Suspicious / Malicious) and priority.
- Deliver enriched alerts to an analyst channel (Discord) in real time.
- Trigger the pipeline automatically from Wazuh for high-signal alerts, and support on-demand enrichment for high-volume telemetry.
- Validate the pipeline end-to-end against a controlled, realistic attack simulation.

3. Development Approach & Honesty Note

I want to be clear about how this project was built, because it reflects where the real work was.

On the implementation: The Python code for the pipeline was written with AI assistance, used as a tool to implement my design quickly (similar to using a library or reference code). The engineering that makes this project mine is the part that matters in a SOC role: the system design, the choice of which event types and threat-intelligence sources to use, the verdict/scoring logic, the Wazuh integration, the troubleshooting (e.g. the rule-level tuning described later), and all of the setup, testing and validation. Every design decision and every test in this document was made and carried out by me.

4. Lab Environment & Tools

Component	Role
Wazuh Server (Ubuntu 24.04 ARM64, 192.168.64.6)	SIEM; hosts the pipeline script and the Wazuh integration
Windows 11 endpoint (DESKTOP-4VD18QN, 192.168.64.4)	Generates the telemetry (Sysmon + Windows Security)
Python 3 (requests, json, ipaddress)	The enrichment pipeline
AbuseIPDB API	IP reputation, geolocation, abuse score
VirusTotal API	File-hash reputation (engine detections)
Discord webhook	Delivery channel for enriched alert cards
Windows Defender	Native AV; its detections are also triaged by the pipeline

5. How the Pipeline Works

The pipeline has four stages, run as a single Python script on the Wazuh server. The first sentence of each alert's life is: Wazuh raises an alert, the script receives it, and the analyst ends up with an enriched card.

```
Wazuh alert  ->  (1) Parse  ->  (2) Enrich  ->  (3) Score  ->  (4) Notify (Discord)
```

1. Parse - read the alert JSON and extract the useful fields (event type, rule, MITRE, process, user, destination IP, file hash).
2. Enrich - look up indicators against threat intel: destination IPs via AbuseIPDB (reputation + country + ISP), file hashes via VirusTotal (engine detections). Private/internal IPs are recognised and skipped.
3. Score - combine the rule severity and the enrichment results into a verdict (Informational / Suspicious / Malicious) and a priority (Low / Medium / High).
4. Notify - format a colour-coded card (green / orange / red) and POST it to the Discord webhook.

API keys and the webhook URL are kept in a separate credentials file, out of the code, so no secrets are hard-coded. This was a deliberate choice to follow good practice.

6. Enrichment Sources & Verdict Logic

Two free threat-intelligence sources were integrated, each validated independently before being wired into the pipeline:

- AbuseIPDB - for a public destination IP, returns an abuse-confidence score, country, ISP and number of reports. Tested against a known public IP (Google DNS) to confirm correct responses.
- VirusTotal - for a file hash, returns how many engines flag it and the threat label. Validated against the EICAR test file's hash, which returned 60+ malicious detections.

The scoring logic I defined maps evidence to a verdict:

Condition	Verdict	Priority
VirusTotal flags the hash, or AbuseIPDB score >= 50, or a Defender detection	MALICIOUS	High
AbuseIPDB score >= 10, or Wazuh rule level >= 12	SUSPICIOUS	High
Wazuh rule level >= 7	SUSPICIOUS	Medium
Otherwise	INFORMATIONAL	Low

7. Wazuh Integration (Automatic Triggering)

The pipeline is wired into Wazuh using its native integration mechanism, so it runs automatically when a qualifying alert fires. A key point: this integration runs on the Wazuh server (the manager side), where the alerts already are. This is more reliable than an agent-side action and, unlike the Active Response in Project 1, the automatic dispatch worked first time here.

The integration is configured to fire on alerts at or above a severity threshold; the script then decides whether the event is one it should triage. High-volume telemetry (process and network events) is handled on-demand instead, as explained in the scope section.

8. Validation Simulation

To prove the pipeline end-to-end, I ran a controlled attack simulation on the Windows endpoint. Each stage is a real attacker action that generates one telemetry type; the pipeline then triaged and enriched it. The cards below are the live output in Discord.

8.1 Stage 1 - Payload dropped to disk (Sysmon 11)

An executable was dropped into the Downloads folder. Wazuh raised a file-creation alert; the pipeline triaged it automatically.



Figure: Sysmon 11 file-creation alert triaged automatically (rule 92203, MITRE T1105)

8.2 Stage 2 - Payload executed (Sysmon 1) + VirusTotal

The dropped file was executed. The process-creation event carries the file's hash, which the pipeline looked up in VirusTotal (a clean result here, a legitimate process).

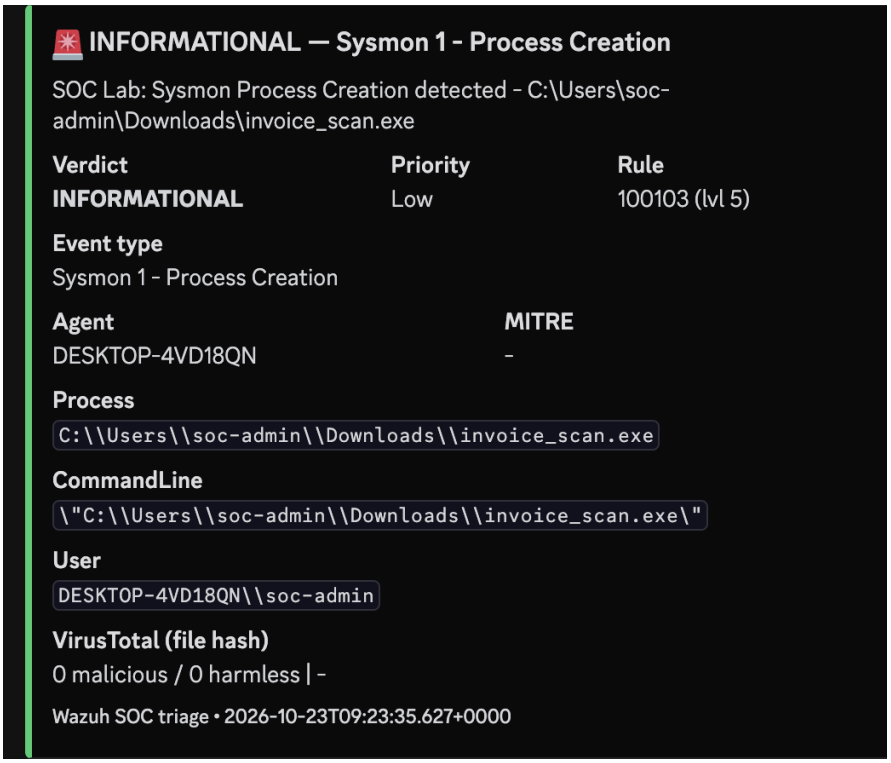


Figure: Sysmon 1 process-creation enriched with a VirusTotal hash lookup

8.3 Stage 3 - Outbound connection (Sysmon 3) + AbuseIPDB

A process made an outbound connection to a public IP. The pipeline enriched the destination IP via AbuseIPDB, returning country, ISP and abuse score. (The card shows the IP, not a domain name, because network-connection events record only the IP; the domain lives in DNS Event 22.)

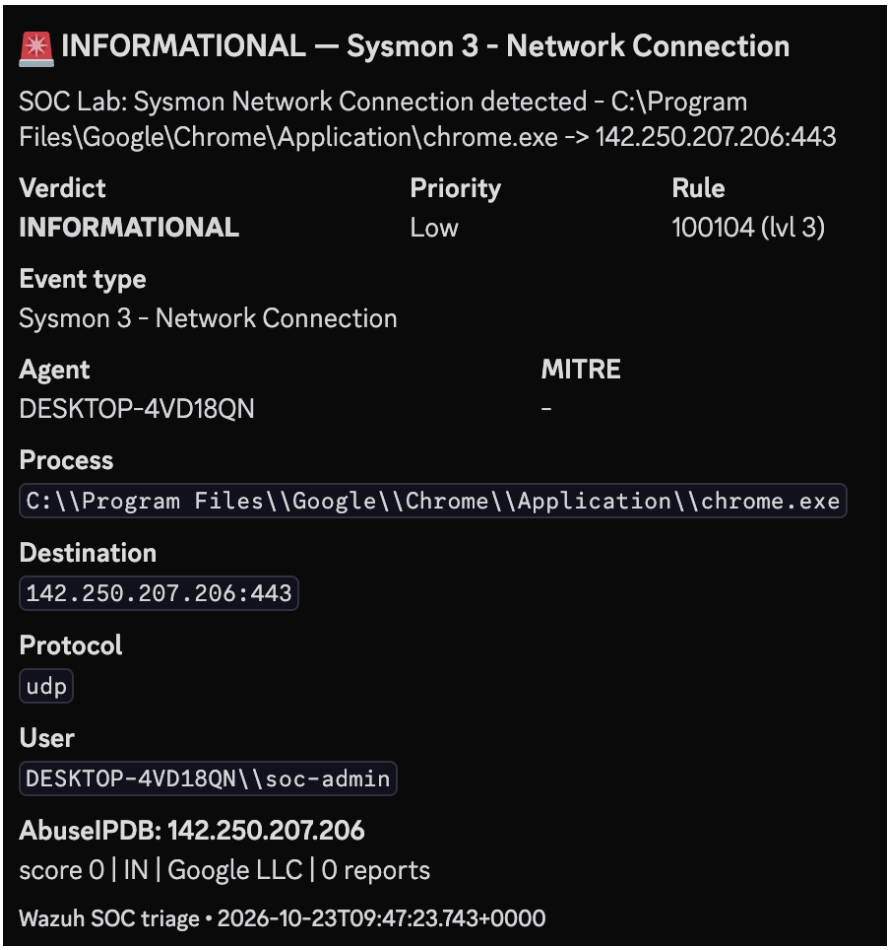


Figure: Sysmon 3 network connection enriched with real AbuseIPDB data

8.4 Stage 4 - Backdoor account created (Windows 4720)

A new local account was created. The pipeline triaged it automatically, showing the new account and who created it.



Figure: Windows 4720 account-creation triaged automatically (MITRE T1098)

8.5 Stage 5 - Privilege escalation (Windows 4732)

The backdoor account was added to the Administrators group. This high-severity event (level 12) was triaged automatically as Suspicious / High.

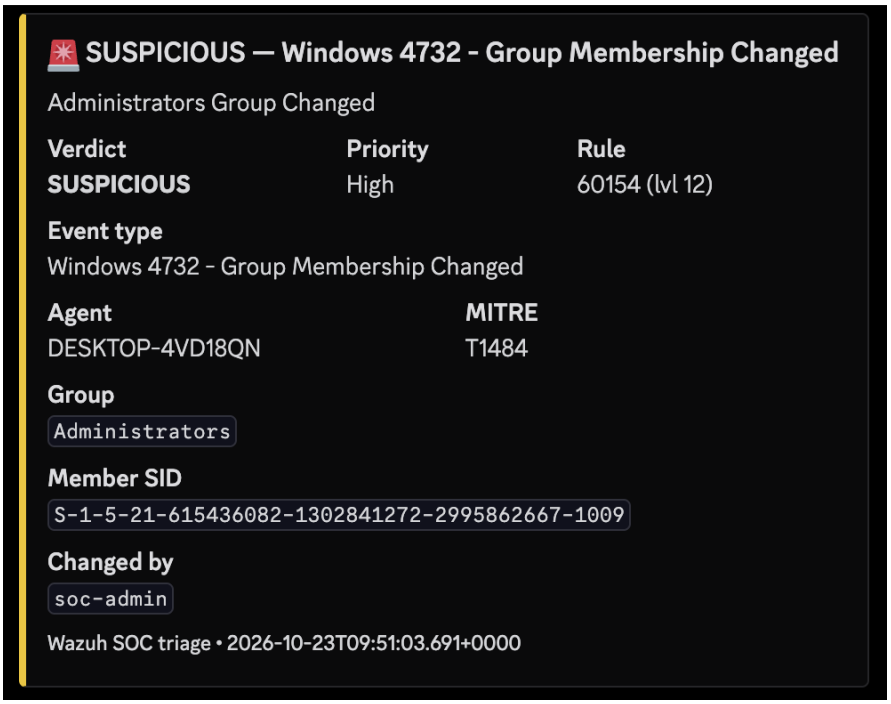


Figure: Windows 4732 privileged-group change triaged automatically (MITRE T1484)

8.6 Stage 6 - Malware detection (Windows Defender)

Finally, the EICAR test file was written to disk, triggering a real Windows Defender detection. I extended the pipeline to treat AV detections as Malicious, producing a genuine red Malicious card from a live Windows event.



Figure: A real Windows Defender detection triaged as MALICIOUS

9. Scope & Design Decisions

The validation above was deliberately scoped, and the scoping itself was a design decision, not a limitation of effort.

- Why these event IDs: I chose Sysmon 1/3/11 and Windows 4720/4732 (plus Defender) because together they cover the core attacker behaviours - run a process, connect out, drop a file, create an account, escalate privilege, and get caught by AV. That gives a representative spread across both Sysmon and Windows Security telemetry without testing hundreds of rules.
- Auto vs on-demand: process and network events (Sysmon 1 and 3) fire constantly on a live endpoint. Auto-enriching every one of them would flood the channel and burn API quota - which is exactly why real SOCs do not do it. So I auto-triage the higher-signal, lower-volume events (file drop, account, group change, AV detection) and enrich the high-volume ones (process, network) on-demand. This is a realistic triage model, not a shortcut.
- Threshold tuning: during testing I found the file-drop fired rule 92203 at level 6, below my initial level-8 trigger, so no card appeared. I traced this and lowered the trigger to level 6 to include file-creation. This kind of tuning is normal detection-engineering work.

- Noise control: I kept the test controlled on purpose. Deeper, high-volume testing is possible but would add noise without adding proof; the goal of this phase was to validate correctness across telemetry types, not to stress-test.

10. What the Pipeline Can Do Further

The current build is a validated foundation. It is designed to extend, and natural next steps include:

- More indicators: enrich domains from DNS (Sysmon Event 22) via VirusTotal/URL reputation, and user/host context.
- More threat-intel sources: add sources such as OTX, GreyNoise or internal block-lists, and cross-reference them.
- Auto-ticketing: write an incident ticket (e.g. to a tracker or a JSON case file) in addition to the Discord card.
- Response actions: move from enrichment toward action - e.g. auto-disable a flagged account or block an IP on a confirmed-malicious verdict (the Project 1 Active Response work feeds directly into this).
- Scale & hygiene: de-duplication, alert grouping, API rate-limit handling, and tunable per-event thresholds for a noisier, production-like environment.

11. Limitations

- Lab indicators (private IPs, a custom test payload) are not in public threat feeds, so the malicious-verdict paths were validated with known indicators (EICAR's hash, a real public IP, a live Defender detection) rather than live unknown malware.
- The pipeline currently enriches IPs and hashes; domains and URLs are future work.
- Testing was scoped to a controlled set of event types (by design); broad, high-volume validation was out of scope for this phase.
- The Python was written with AI assistance (see Section 3); the design, integration and testing are my own.

12. Learning Outcomes

- Designing an end-to-end alert triage and enrichment workflow (SOAR-lite).
- Integrating external threat-intelligence APIs (AbuseIPDB, VirusTotal) and interpreting their results.
- Defining verdict/priority logic that turns raw alerts into analyst-ready decisions.
- Wiring automation into a SIEM via native integrations, and understanding why server-side automation is more reliable.
- Making deliberate scoping and noise-control decisions, and tuning detection thresholds from real findings.
- Keeping secrets out of code and validating each component before integration.

13. Conclusion

This project turns the Project 1 SIEM from something that raises alerts into something that triages them. I designed a pipeline that automatically enriches Wazuh alerts with real threat intelligence, scores them into clear verdicts, and delivers them to an analyst channel, and I validated it end-to-end against a realistic six-stage attack covering Sysmon and Windows telemetry and a live AV detection. The result demonstrates the core Tier-1 SOC workflow - detect, enrich, triage, prioritise - built and tested by me, with the implementation accelerated using AI assistance.